

AI-Powered API Performance Analytics Platform

An event-driven microservices platform for real-time API observability with machine-learning-based predictive monitoring.

Java 21

Spring Boot 3

Spring Cloud Gateway

Apache Kafka

PostgreSQL 15

Redis 7

Python 3.11

FastAPI

scikit-learn

XGBoost

Docker Compose

Repository: github.com/arckit11/ai-api-observability-platform

Local path: /Users/macbookpro/Desktop/innovations_API

Status: [Phases 0 · 1 · 2 · 3 · 4 · Frontend · Alerting complete](#)

Table of contents

1. What this project is
2. How it works — architecture and data flow
3. What each service does
4. Machine learning subsystem
5. Running the demo — step-by-step
6. What to point at during the walkthrough
7. Tear-down and troubleshooting

1. What this project is

Modern distributed systems rely on dozens of interdependent APIs whose failures can cascade rapidly across an entire platform. Existing observability tools are predominantly *reactive*: they report what has happened, not what is *about to* happen. Threshold alerts only fire once a metric has already breached a boundary, and commercial platforms (Datadog, New Relic) carry licensing costs that put them out of reach for many teams.

This project is a working reference implementation of an **AI-powered API performance analytics platform**. It combines a Java/Spring Boot event-driven backend with a Python/FastAPI machine learning subsystem to provide proactive, intelligent API monitoring.

The key idea. The gateway captures request telemetry and streams it through Kafka. Analytics computes rolling aggregates. Five ML modules (traffic forecasting, failure prediction, anomaly detection, composite health scoring, and alert prioritization) turn those aggregates into predictions that anticipate degradation before it hits users. The dashboard fans these signals into a single view with a circuit-breaker fallback so ML outages never take the dashboard down.

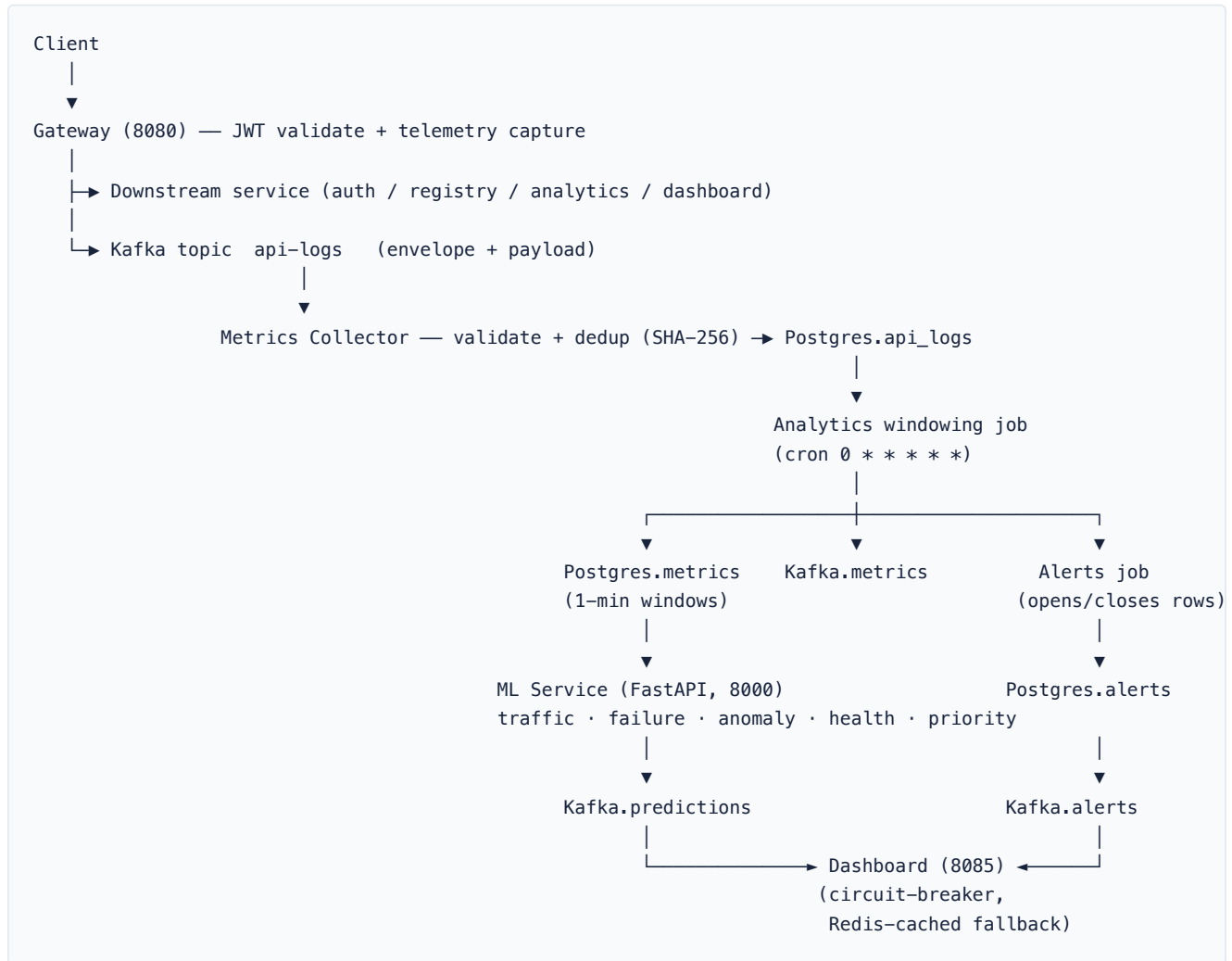
Highlights

- **Contract-first.** OpenAPI + Kafka event schemas are the source of truth between both subsystems.

- **Event-driven.** Every request captured at the gateway is streamed via Kafka. Downstream services never block the hot path.
- **Predictive, not reactive.** Five ML modules are first-class REST endpoints, not add-ons.
- **Resilient by design.** Dedup keys absorb Kafka at-least-once duplicates, a DLQ isolates poison messages, and a circuit breaker fronts the ML calls.

2. How it works — architecture and data flow

End-to-end path of a single API request



Two channels, everywhere

- **Synchronous REST (HTTP/JSON)** for request–response calls between services.
- **Asynchronous Kafka topics** for telemetry, metrics, predictions and alerts. Six topics: `api-logs`, `api-logs-dlq`, `metrics`, `alerts`, `predictions`, `service-health`.

Postgres schema (7 tables)

Table	What it holds
users	Local accounts with role: ADMIN / DEVELOPER / VIEWER
services	API registry — every monitored service
api_logs	Raw request telemetry. Unique index on dedup_key for at-least-once safety.
metrics	1-minute aggregates: RPM, latency percentiles, error rate. Range-partitioned by window_start .
alerts	Threshold-breach alerts opened by Analytics; ML enriches with a priority label.
predictions	Every prediction emitted by an ML module (JSONB value).
ml_model_results	One row per training run: algorithm, metrics, artifact path, is_active flag.

3. What each service does

Gateway service · :8080

Spring Cloud Gateway (reactive). Validates JWTs, enforces role-based access control, routes to the right downstream, and publishes one `api-log` envelope per forwarded request. Open paths: `/auth/login` , `/auth/refresh` , `/actuator/*` .

Auth service · :8081

Issues and refreshes JWTs (HS256). Passwords stored as BCrypt hashes. Bootstrap admin seeded by migration V5 — user `admin` , password `admin1234!` .

Registry service · :8082

CRUD over the `services` table. Guards against (name, environment) collisions. Returns RFC 7807 problem details on error.

Metrics collector · :8083

Kafka consumer for `api-logs` . Validates payload, dedupes via `dedup_key` , writes to Postgres. Malformed events go to `api-logs-dlq` instead of blocking the partition.

Analytics service · :8084

Two scheduled jobs. **Windowing** closes the previous 1-min window, aggregates `api_logs` with SQL `percentile_cont` for P95/P99, upserts into `metrics` , fans results to Kafka. **Alerting** opens rows on `P99 > 2 s` or `error_rate > 10%` and auto-closes them. Redis-cached read APIs.

Dashboard service · :8085

Read-side aggregator. Every tile call fans out to Analytics + ML. ML calls are wrapped in a Resilience4j circuit breaker; on open circuit the service returns the last cached prediction from Redis (marked `stale: true`).

ML service · :8000 (Python/FastAPI)

Serves the five predictive modules. Loads trained joblib bundles on startup; publishes every prediction to the `predictions` Kafka topic. Includes an `/admin/models` endpoint that lists loaded artifacts and a `/admin/models/<module>/reload` endpoint for hot-reloading.

4. Machine learning subsystem

Endpoint	Module	Algorithm	What it does
POST <code>/predict/traffic</code>	Traffic forecasting	XGBoost Regressor (per-horizon)	Predict RPM at $h \in \{15, 30, 60, 360\}$ minutes ahead.
POST <code>/predict/failure</code>	Failure prediction	XGBoost Classifier	$P(\text{failure in next 30 min})$. Recall-weighted threshold picked at fit time.
POST <code>/predict/anomaly</code>	Anomaly detection	Isolation Forest	Unsupervised outlier detection with contamination auto-tuned against synthetic ground truth.
POST <code>/score/health</code>	Composite health score	Weighted linear	$H = 100 \cdot (0.30 \cdot \hat{e} + 0.30 \cdot \hat{f} + 0.20 \cdot \hat{t} + 0.20 \cdot \hat{r})$. Missing signals renormalise gracefully.
POST <code>/alerts/prioritize</code>	Alert prioritization	Random Forest	Multi-class classifier: Low / Medium / High / Critical.

Feature engineering (paper Table II)

Request count · response time (mean, P95, P99) · error rate · CPU / memory usage · cyclical time encodings (sin/cos of hour and day-of-week) · engineered lag features (RPM at t-1h, t-6h, t-24h) · rolling 15-minute mean.

Training results on synthetic 30-day data

Module	Metrics
Traffic	MAPE 7–9 % across horizons 15 / 30 / 60 / 360 min
Failure	Precision 0.28, Recall 0.29, F1 0.29, ROC-AUC 0.71
Anomaly	Contamination 0.05 (auto-tuned), Recall 0.44
Alerts	Macro F1 0.73, accuracy 96 % across four classes

5. Running the demo — step-by-step

Step 0 · Prerequisites

Docker Desktop running, `mvn` and `python3` on the PATH. Quick check:

```
docker info >/dev/null && echo "docker ok"
mvn --version | head -1
python3 --version
```

Step 1 · Set up the shell environment

```
export JAVA_HOME=/opt/homebrew/Cellar/openjdk/26.0.1/libexec/openjdk.jdk/Contents/Home
export PATH="$JAVA_HOME/bin:$PATH"
export JWT_SECRET="local-dev-secret-min-32-chars-required-here"
export POSTGRES_HOST=localhost POSTGRES_PORT=5432
export REDIS_HOST=localhost REDIS_PORT=6379
export KAFKA_BOOTSTRAP_SERVERS=localhost:29092
export AUTH_HOST=localhost REGISTRY_HOST=localhost \
  ANALYTICS_HOST=localhost DASHBOARD_HOST=localhost ML_HOST=localhost
cd /Users/macbookpro/Desktop/innovations_API
```

Step 2 · Start infrastructure

```
make up
docker compose ps      # all five containers should be Up
                        # (kafka-init runs once and exits 0 – that is fine)
```

Open <http://localhost:8090> for the Kafka UI.

If Kafka fails on boot with "NodeExists":

```
docker compose restart zookeeper
docker compose up -d kafka
```

Step 3 · Build the backend (one-time)

```
cd backend && mvn -DskipTests install && cd ..
```

Step 4 · Generate synthetic data (optional — only if you want fresh data)

```
cd synthetic-data
python3 -m venv .venv && source .venv/bin/activate
pip install -q -e .
synth clear --since 2020-01-01
synth generate --days 30 --seed 42      # ~216k rows across 5 services
deactivate && cd ..
```

Step 5 · Train the ML models (only if `ml-service/app/models/*.joblib` is missing)

```
cd ml-service
python3 -m venv .venv && source .venv/bin/activate
pip install -q -e ".[dev]"
export ML_MODEL_DIR="$(pwd)/app/models"
python -m app.training.cli --days 30
deactivate && cd ..
```


Step 6 · Start all seven services

```
JAVA="$JAVA_HOME/bin/java"

nohup "$JAVA" -jar backend/auth-service/target/auth-service-0.1.0-SNAPSHOT.jar > /tmp/auth.log 2>&1 &
nohup "$JAVA" -jar backend/registry-service/target/registry-service-0.1.0-SNAPSHOT.jar > /tmp/registry.log 2>&1 &
nohup "$JAVA" -jar backend/metrics-collector/target/metrics-collector-0.1.0-SNAPSHOT.jar > /tmp/collector.log 2>&1 &
nohup "$JAVA" -jar backend/gateway-service/target/gateway-service-0.1.0-SNAPSHOT.jar > /tmp/gateway.log 2>&1 &
nohup "$JAVA" -jar backend/analytics-service/target/analytics-service-0.1.0-SNAPSHOT.jar > /tmp/analytics.log 2>&1 &
nohup "$JAVA" -jar backend/dashboard-service/target/dashboard-service-0.1.0-SNAPSHOT.jar > /tmp/dashboard.log 2>&1 &

cd ml-service && source .venv/bin/activate
export ML_MODEL_DIR="$(pwd)/app/models"
nohup uvicorn app.main:app --host 0.0.0.0 --port 8000 > /tmp/ml.log 2>&1 &
deactivate && cd ..

sleep 20
```

Sanity: `curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8080/auth/login` should return 400 (endpoint alive, body missing).

Step 7 · Seed a couple of live alerts so the alerts tile is not empty

```
docker compose exec -T postgres psql -U api_analytics -d api_analytics <<'SQL'
INSERT INTO metrics (service_id, window_start, window_end, window_size,
                    request_count, rpm, response_time_mean, response_time_p95,
                    response_time_p99, error_rate, success_rate, computed_at)
VALUES
    ('11111111-1111-1111-1111-111111111111',
     date_trunc('minute', NOW()), date_trunc('minute', NOW()) + interval '1 minute',
     '1m', 500, 500, 250, 800, 4500, 0.15, 0.85, NOW()),
    ('33333333-3333-3333-3333-333333333333',
     date_trunc('minute', NOW()), date_trunc('minute', NOW()) + interval '1 minute',
     '1m', 200, 200, 400, 900, 2100, 0.18, 0.82, NOW())
ON CONFLICT (service_id, window_size, window_start) DO UPDATE
SET response_time_p99 = EXCLUDED.response_time_p99,
    error_rate = EXCLUDED.error_rate,
    computed_at = NOW();
SQL
```

The alerting cron fires at second `:05` of each minute — wait ~60 s and the two rows appear as alerts.

Step 8 · Start the frontend

```
cd frontend
python3 -m http.server 5173
```

Open **`http://localhost:5173`** in your browser.

6. What to point at during the walkthrough

1. **Sign in.** Top bar: `admin` / `admin1234!`. The chip flips to green "signed in".
2. **Service health tile.** One card per service. The big number is the ML-computed composite score (0-100); the label under it is the derived status (healthy / warning / degraded / critical).
3. **Traffic tile.** Current RPM per service on the left, 60-min forecast on the right. The Δ column shows the direction — a red $\uparrow$ means predicted growth, a green $\downarrow$ means predicted decline.
4. **Latency tile.** Mean, P95, P99 for the latest window, straight from Analytics — no ML involved. Good baseline for comparing against the predictions tile.
5. **Predictions tile.** Anomaly score (0-1) plus a boolean flag, then failure probability with a discrete risk label. Both come from the trained XGBoost / Isolation Forest models.
6. **Open alerts tile.** Rows opened by the Analytics threshold job (raw `severity`) enriched with an `ml_priority` label from the Random Forest classifier. This is where the two subsystems visibly meet.
7. **Kafka UI.** Switch to `http://localhost:8090` and open the `predictions` or `alerts` topic → Messages. You will see the exact envelope shape defined in `docs/api-contracts/kafka-events.md`, streaming live as you refresh the dashboard.

Curl-only version (if you need to demo without a browser)

```
TOKEN=$(curl -s -X POST http://localhost:8080/auth/login \
-H 'content-type: application/json' \
-d '{"username":"admin","password":"admin1234!"}' \
| python3 -c "import json,sys; print(json.load(sys.stdin)['access_token'])")

for tile in health traffic latency predictions alerts; do
  echo "=== /dashboard/$tile ==="
  curl -s -H "Authorization: Bearer $TOKEN" http://localhost:8080/dashboard/$tile \
  | python3 -m json.tool | head -20
  echo
done
```

7. Tear-down and troubleshooting

Stop everything

```
kill -f "java -jar backend/" 2>/dev/null
kill -f "uvicorn app.main" 2>/dev/null
kill -f "python3 -m http.server 5173" 2>/dev/null
make down # stop containers, keep the database
# make clean # only if you want to wipe volumes (fresh start)
```

Common gotchas

Symptom	Fix
Kafka won't start after reboot ("NodeExists")	<code>docker compose restart zookeeper && docker compose up -d kafka</code>
<code>/readyz</code> shows <code>models_loaded: []</code>	Run step 5 — the joblib files are gitignored, models must be trained locally.
Alerts tile is empty	Run step 7 and wait ~60 seconds for the cron.
CORS error in the browser	Serve the frontend via <code>python3 -m http.server</code> ; don't open <code>index.html</code> with <code>file://</code> . Gateway CORS is set to <code>*</code> .
<code>413 Unauthorized</code> on protected endpoints	The JWT expired (15 min default). Click <i>Log in</i> again.

Everything above is committed to `github.com/arckit11/ai-api-observability-platform` . The five commits map to the platform's roadmap phases: infra scaffold, front-half pipeline, ML models, Analytics windowing + Dashboard aggregation, and the ML-tuning + alerting + frontend follow-up.